Apache Kafka Operations and Troubleshooting

1. Kafka Operations Overview

Operating Kafka requires monitoring brokers, partitions, consumer groups, replication health, disk usage, throughput, latency, and consumer lag.

2. Consumer Lag

Consumer lag represents how far a consumer group is behind the latest available records. Persistent lag can indicate insufficient consumer capacity, slow downstream processing, blocked consumers, or a traffic spike.

High lag should be correlated with throughput, processing time, partition assignments, and downstream dependencies.

3. Retention

Kafka can retain records for a configured time period, size limit, or both. Retention is not the same as a traditional archive policy. Records may be deleted when retention rules are reached.

Choose retention based on recovery requirements, replay needs, storage capacity, and compliance constraints.

4. Partition Scaling

Adding partitions can increase parallelism but can affect key distribution and ordering assumptions. Partition count should therefore be treated as an architectural decision rather than changed casually.

Changing partition count can alter how keyed records are distributed for future writes.

5. Replication and ISR

The in-sync replica set contains replicas considered sufficiently caught up according to Kafka's configuration. Monitoring ISR shrinkage can reveal broker or network problems.

Under-replicated partitions are an important operational warning because they indicate reduced redundancy.

6. Common Failure Patterns

Common Kafka problems include broker disk exhaustion, network interruptions, under-replicated partitions, consumer lag, bad serialization, authentication failures, authorization errors, and overloaded consumers.

7. Producer Reliability

Useful producer settings and design concepts include acknowledgments, retries, idempotence, delivery timeout, batching, compression, and maximum request size.

Reliability settings should be selected with latency and throughput requirements in mind.

8. Consumer Reliability

Consumer applications should handle retries, poison messages, deserialization errors, downstream failures, and graceful shutdown. Dead-letter topics can isolate records that repeatedly fail processing.

Commit offsets at a point that matches the desired processing semantics.

9. Monitoring Checklist

Monitor broker CPU, memory, disk, network throughput, request latency, under-replicated partitions, offline partitions, consumer lag, message rates, and error rates.

Alert thresholds should reflect normal workload patterns rather than arbitrary universal numbers.

10. Troubleshooting Example

Scenario: a Spark streaming job has increasing Kafka lag. First confirm whether input traffic increased. Then check Spark batch duration, Kafka partition count, executor utilization, downstream sink latency, and failed micro-batches.

A lag problem is not automatically a Kafka broker problem; the bottleneck may be the consumer application or downstream system.